Operating System Fundamentals • Chapter 02

Computer System Structure

BIG QUESTION

You press a key. How does the letter reach the screen?

What happens inside the box: devices, interrupts, I/O, storage, and how the hardware protects the OS.

Chapter 2 of the notebook

Today's route

What we'll cover today

1. **System operation**
   Buses, controllers, and startup
2. **Interrupts**
   How hardware gets the CPU's attention
3. **I/O structure**
   Synchronous, asynchronous, DMA
4. **Storage structure**
   From registers to tape
5. **Hardware protection**
   Stopping one program from breaking the others

Part 01

Computer system operation

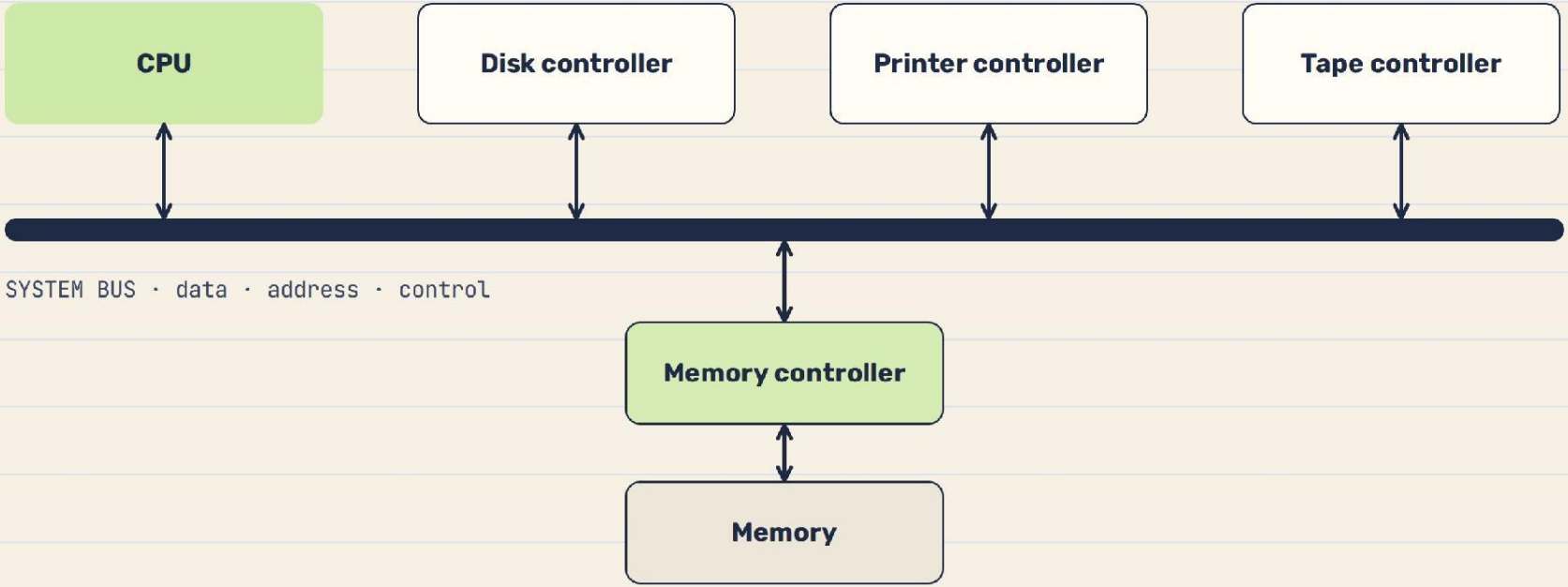
THINK FIRST

When you click Print, does the CPU stand next to the printer until the page comes out?

discuss for one minute before the next slide

The layout

Everything shares one common bus



How they cooperate

Controllers work while the CPU works

Runs concurrently

The CPU and every device controller execute at the same time, competing for memory cycles.

One controller, one device type

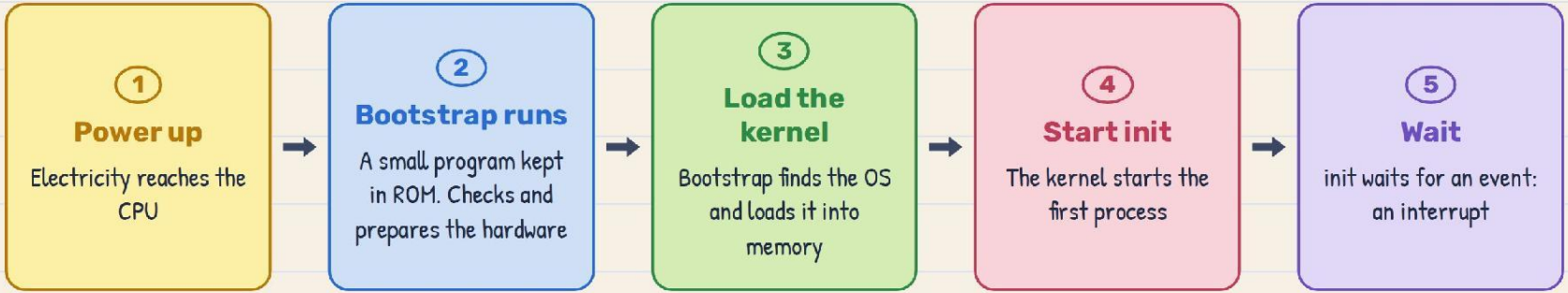
Each controller has a local buffer. The CPU moves data between main memory and those buffers.

"I'm done!"

When a controller finishes its operation, it tells the CPU by raising an interrupt.

Computer system startup

From power button to login screen



Why ROM? It keeps its contents with the power off, so the very first program is always there.

Part 02

Interrupts

THINK FIRST

Your phone doesn't check every second whether a message arrived; it gets tapped on the shoulder. How does the CPU get tapped?

discuss for one minute before the next slide

Interrupts

A signal that says: stop, deal with me



FROM DEVICES

Hardware interrupt

- Sent by a device over the bus
- e.g. the disk finished reading
- e.g. a key was pressed



FROM PROGRAMS

Software interrupt = trap

- **Error:** division by zero, invalid memory access
- **Request:** a user program asks the OS for a service (a *system call*)

System calls in detail → Chapter 3

An operating system is interrupt driven.

No interrupts, no work. Every keystroke, disk read, timer tick and system call reaches the OS as an interrupt.

Interrupt handling

Five steps, in a fraction of a microsecond

1. CPU is interrupted
2. CPU stops the current process
3. CPU jumps to a fixed location
4. CPU runs the interrupt service routine
5. CPU resumes the process



Handle it quickly

Interrupts arrive constantly. A slow handler slows the whole machine.



Save the state first

Store the interrupted process's information so it resumes exactly where it stopped.

Finding the right handler

Generic routine vs interrupt vector

OPTION 1

Generic handler

One routine looks at the interrupt, works out who raised it, then calls the right routine. Simple but slower.

OPTION 2

Interrupt vector

A ready-made list: interrupt number → its handling routine. Jump straight there. Used in UNIX and DOS.



Real life

A hotel key rack: the room number takes you straight to the right key, no searching.

Pop quiz • click to reveal

A program divides a number by zero. What happens first?

- **A** The disk controller raises an interrupt
- **B** The hardware raises a trap to the OS
- **C** DMA copies the result to memory
- **D** The bootstrap program restarts

Pop quiz • click to reveal

A program divides a number by zero. What happens first?

A. The disk controller raises an interrupt

B. The hardware raises a trap to the OS

C. DMA copies the result to memory

D. The bootstrap program restarts

Answer B. An error inside a program is a software-generated interrupt: a trap. The OS then decides what to do with the process.

Part 03

I/O structure

THINK FIRST

Copying a 4 GB movie to a USB stick: should the CPU carry it across one byte at a time?

discuss for one minute before the next slide

Device controllers

The middle-man for every device



Specialist

In charge of one specific type of device.



Mover

Moves data between its device and its local buffer.



Can share

One controller may handle more than one device.



Buffer size varies

From a few bytes to many megabytes, depending on the device.

Two I/O methods

Wait for it, or carry on?

Synchronous: control returns only when I/O completes

Process	runs	waits…	runs
Device	idle	I/O transfer	idle

Asynchronous: control returns immediately, I/O continues

Process	runs	keeps running	runs
Device	idle	I/O transfer	idle

Analogy: waiting at the counter for your coffee (synchronous) vs taking a buzzer and going back to your table (asynchronous).

Direct Memory Access (DMA)

Move whole blocks without the CPU

THE SLOW WAY

Without DMA

- CPU copies data from the controller's buffer
- One interrupt per byte (or word)
- CPU is busy the whole time

THE FAST WAY

With DMA

- For high-speed devices, close to memory speed
- Controller moves a block from its buffer straight into main memory
- One interrupt per block

The CPU only sets up the transfer and handles one interrupt at the end. In between it runs other work.

Pop quiz • click to reveal

Why is DMA faster for a disk transfer?

A It makes the disk spin faster

B It raises one interrupt per block instead of per byte

C It skips main memory entirely

D It turns off interrupts

Pop quiz • click to reveal

Why is DMA faster for a disk transfer?

A. It makes the disk spin faster

B. It raises one interrupt per block instead of per byte

C. It skips main memory entirely

D. It turns off interrupts

Answer B. The controller copies the whole block into memory itself and interrupts the CPU once when it is done.

Part 04

Storage structure

THINK FIRST

Why doesn't your computer use one kind of memory for everything?

discuss for one minute before the next slide

Two kinds of storage

Main memory and secondary storage



Main memory (RAM)

The only large storage the CPU can access directly. Fast, but volatile: it loses everything when power is lost.

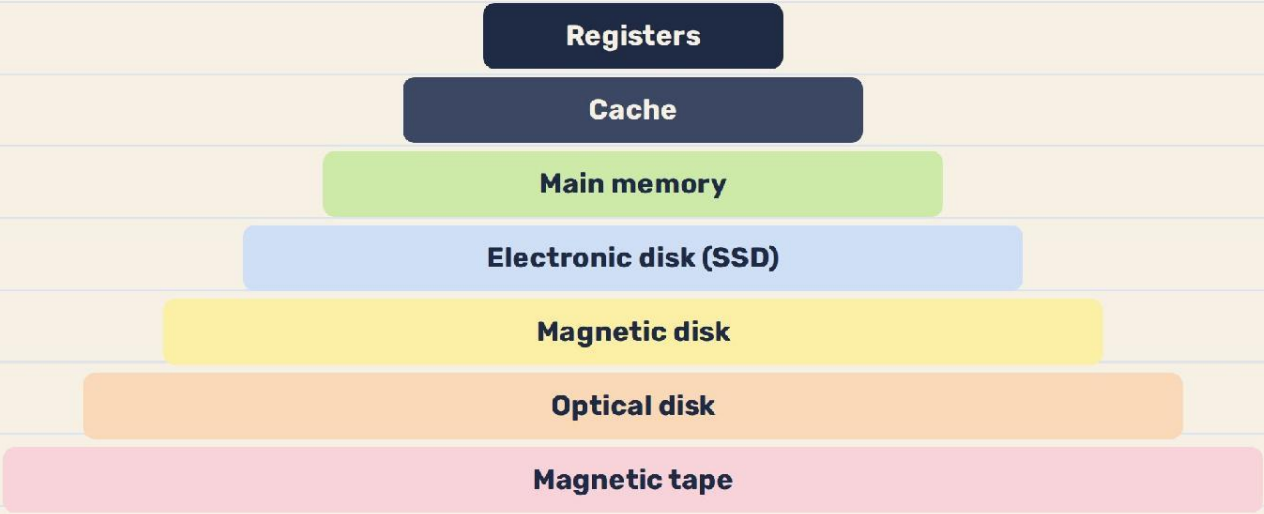


Secondary storage

An extension of main memory that gives large, non-volatile capacity. Disks are the main example.

Storage-device hierarchy

Fast and small on top, big and cheap below



↑ Up: faster, more expensive per byte, smaller, volatile

↓ Down: slower, cheaper, larger, non-volatile

Comparing storage devices

Six questions to ask any device



Speed

How fast can I read and write?



Cost

How much per byte?



Capacity

How much can it hold?



Volatility

Does it forget when power is off?



Reliability

How likely is it to fail?



Portability

Can I carry it around?

Main memory up close

An array of words, every byte with an address



Addresses

Every byte in memory has its own address, like house numbers on a long street.



Load and store

Load brings data from memory into the CPU. **Store** puts it back into memory.

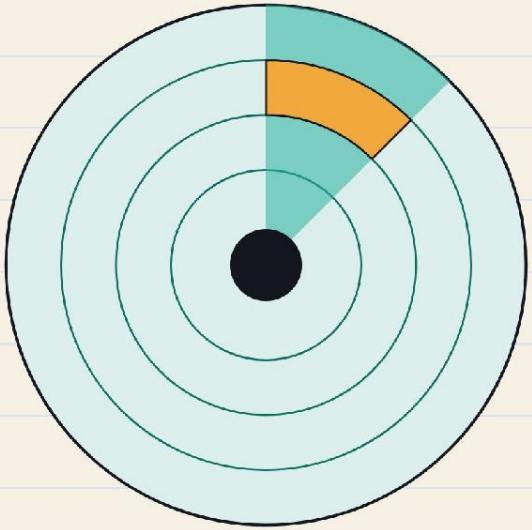


Two kinds of address

Physical and **logical**. We meet the difference in Chapter 7.

Magnetic disk

Platters, tracks and sectors



Platters

Rigid metal or glass disks covered with magnetic recording material.

Tracks → sectors

The surface is logically divided into circular tracks, which are subdivided into sectors (orange).

Disk controller

Decides the logical interaction between the device and the computer.

Magnetic tape

The oldest secondary storage is still around



Early secondary storage

One of the first ways to keep data outside main memory.



Slow access time

Sequential: to reach the end you must wind past everything before it.



Still useful for

- Backups
- Storing information that is rarely used

Pop quiz • click to reveal

Which storage loses its contents when the power goes off?

- **A** Magnetic disk
- **B** Main memory (RAM)
- **C** Magnetic tape
- **D** Optical disk

Pop quiz · click to reveal

Which storage loses its contents when the power goes off?

A Magnetic disk

B Main memory (RAM)

C Magnetic tape

D Optical disk

Answer B. Main memory is volatile. Everything below it in the hierarchy is non-volatile.

Part 05

Hardware protection

THINK FIRST

If one buggy program could overwrite the operating system, what would happen to every other program?

discuss for one minute before the next slide

Why protection became necessary

One user was fine; many programs were not

THEN

Early OS

- Single user
- Programmer had full control of the hardware
- Programmer was responsible for I/O

THE PROBLEM

An error in a program

- **Single task:** only that program is affected
- **Multi-task:** it can damage other programs
- Early desktop OSes let any program touch any memory

Error handling

Hardware catches it, the OS handles it



Typical errors

- Illegal instruction
- Infinite loop
- Accessing another program's memory

1. Hardware detects the error
2. Hardware traps to the OS
3. OS handles the error
4. OS terminates the process
5. OS dumps the process memory to disk, if needed

The foundation

Two modes: user and monitor



User mode

Where your programs run. Dangerous instructions are not allowed.



Monitor (kernel) mode

Where the OS runs. Every instruction is allowed.

A trap or interrupt switches to monitor mode; the OS switches back before returning to the program. All three protections on the next slide rely on this.

Ensuring the OS runs properly

Three protections, three problems



01

I/O protection

Problem: a program misusing a device. Fix: only the OS may talk to devices; programs must ask it.



02

Memory protection

Problem: illegal memory access. Fix: **base and limit registers** fence each program's memory.



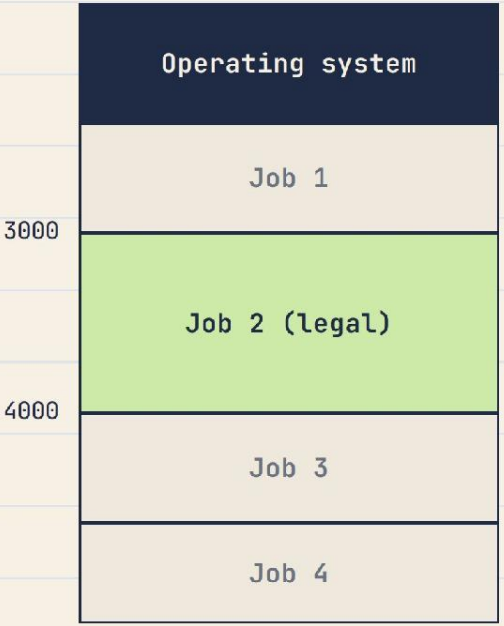
03

CPU protection

Problem: infinite loops. Fix: a **timer** interrupts the program so the OS gets control back.

Memory protection

Base and limit registers



Base register = 3000

Where the program's memory starts.

Limit register = 1000

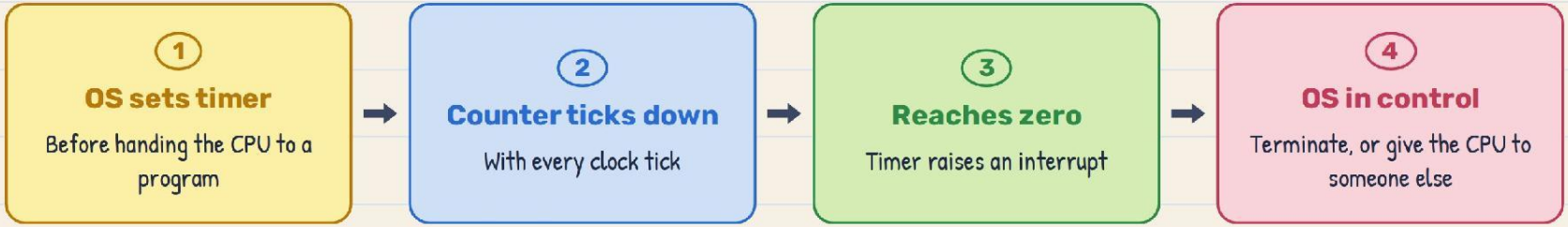
How much memory it owns. Allowed: 3000 up to 3999. Anything else is blocked.

↺ Recall · full mechanism in Ch 7

Chapter 7 shows the hardware check, address by address.

CPU protection

The timer: the OS always gets the CPU back



Without it, a program stuck in an endless loop would own the CPU forever.

Pop quiz • click to reveal

A program gets stuck in an endless loop. Which protection stops it from freezing the machine?

- A Base and limit registers
- B The timer
- C DMA
- D The interrupt vector

Pop quiz • click to reveal

A program gets stuck in an endless loop. Which protection stops it from freezing the machine?

- A. Base and limit registers
- B. The timer
- C. DMA
- D. The interrupt vector

Answer B. The timer interrupt guarantees the OS regains control, even from an infinite loop.

Wrap-up

What to remember

OS JOURNAL · CHAPTER 2

- ✓ CPU and controllers share a bus and run at the same time.
- ✓ The OS is interrupt driven: hardware interrupts and traps.
- ✓ Storage trades speed for size, cost and volatility.
- ✓ The bootstrap in ROM loads the kernel; the kernel starts init.
- ✓ DMA moves whole blocks with one interrupt per block.
- ✓ Dual mode, base/limit registers and the timer keep the OS in charge.

Next up *Chapter 3 · Operating System Structures: what the OS offers programs*